Differential Drive Mobile Robot Simulation and Dead Reckoning

Diego Marzá Fuertes and Mario López Honrubia

October 2026

Abstract

This report presents the implementation and validation of a differential-drive mobile robot simulator and a dead-reckoning localisation method. Planar direct and inverse pose compounding were first implemented and then incorporated into the simulation and localisation modules. The simulator integrates the commanded linear and angular velocities while generating noisy wheel-encoder readings. These measurements are subsequently used to estimate the robot pose without access to the simulated ground truth. The resulting trajectories show the expected accumulation of odometric error: the position error is 1.019 m after the first 5000 simulation steps and reaches 12.481 m after the complete 10000-step sequence. The exercise demonstrates how small encoder errors propagate through successive motion updates and produce increasing drift in dead-reckoning localisation.

1 Introduction

The objective of Labs 1 and 2 is to build the simulation and odometric localisation foundations required by the later probabilistic localisation exercises. The Pre-Lab covered the planar pose representation and the direct and inverse pose-compounding operators. Lab 1 then applied these operations to the differential-drive simulator, wheel-encoder model, and commanded trajectories. Lab 2 used the simulated encoder readings to estimate the robot pose through dead reckoning.

The implementation uses a world frame, denoted by N, and a body-fixed robot frame, denoted by B. A planar robot pose contains two Cartesian coordinates and one yaw angle. Lab 1 produces the simulated or ground-truth state, whereas Lab 2 reconstructs the pose using only incremental encoder information.

The report is organised accordingly. The remainder of this introduction describes the software structure and the Pre-Lab pose operations. Section 2 presents the differential-drive simulation, and section 3 describes the dead-reckoning implementation. The experimental comparison is discussed in section 4, followed by the conclusions in section 5.

1.1 Development environment and software structure

Jupyter Notebook was used only in Lab 0 to implement and validate the direct and inverse pose-compounding operations, $\oplus$ and $\ominus$. These operations were subsequently incorporated into the Python modules used in Labs 1 and 2. NumPy provides vector and matrix operations, Matplotlib is used for visualisation, and Robotics Toolbox supplies the animated vehicle representation.

The relevant software components and their roles are summarised in table 1.

Table 1: Main Python files used in Labs 1 and 2.

File	Responsibility
`Pose3D.py`	Represents a planar pose and implements direct and inverse pose compounding.
`SimulatedRobot.py`	Defines the common simulation interface, state storage and visualisation infrastructure.
`DifferentialDriveSimulatedRobot.py`	Implements the differential-drive motion model and simulated sensors.
`Localization.py`	Provides the iterative localisation loop and trajectory visualisation.
`DR_3DOFDifferentialDrive.py`	Converts encoder readings into body motion and performs dead-reckoning integration.
`main.py`	Configures the map and initial state, creates the simulated and dead-reckoning robots, and runs the localisation sequence.

1.2 Pre-Lab: Planar pose operations

1.2.1 Planar pose representation

A three-degree-of-freedom pose is represented by

$$\boldsymbol{\eta} = \begin{bmatrix} x & y & \psi \end{bmatrix}^T \in \mathbb{R}^3, \tag{1}$$

where (x, y) is the position and ψ is the yaw angle. Its equivalent homogeneous transformation is

$$T(\boldsymbol{\eta}) = \begin{bmatrix} \cos\psi & -\sin\psi & x \\ \sin\psi & \cos\psi & y \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

This matrix form explains the pose operations used throughout the simulator: direct compounding corresponds to multiplying transformations, and inverse compounding corresponds to inverting a transformation.

1.2.2 Direct pose compounding

Let ${}^{A}\boldsymbol{x}_B$ describe frame B relative to frame A, and let ${}^{B}\boldsymbol{x}_C$ describe frame C relative to B. The pose of C relative to A is

$$ {}^{A}\boldsymbol{x}_C = {}^{A}\boldsymbol{x}_B \oplus {}^{B}\boldsymbol{x}_C, \tag{3} $$

which is equivalent to $T({}^{A}\boldsymbol{x}_C) = T({}^{A}\boldsymbol{x}_B)T({}^{B}\boldsymbol{x}_C)$. In coordinate form,

$$ \begin{bmatrix} {}^{A}x_C \\ {}^{A}y_C \\ {}^{A}\psi_C \end{bmatrix} = \begin{bmatrix} {}^{A}x_B + {}^{B}x_C\cos({}^{A}\psi_B) - {}^{B}y_C\sin({}^{A}\psi_B) \\ {}^{A}y_B + {}^{B}x_C\sin({}^{A}\psi_B) + {}^{B}y_C\cos({}^{A}\psi_B) \\ {}^{A}\psi_B + {}^{B}\psi_C \end{bmatrix}. \tag{4} $$

The $\oplus$ operator is used to compose poses or motions expressed in different reference frames. In this case, ${}^{B}\boldsymbol{x}_C$ cannot be added directly to ${}^{A}\boldsymbol{x}_B$ because its components are written in frame B. The operation first transforms them into frame A and then combines both poses to obtain ${}^{A}\boldsymbol{x}_C$.

The implementation extracts the pose components, evaluates equation (4), and returns another `Pose3D` object. The essential operation is:

Listing 1: Core of the direct compounding implementation.

```
x_result = xB + xC * cos(theta_B) - yC * sin(theta_B)
y_result = yB + xC * sin(theta_B) + yC * cos(theta_B)
theta_result = theta_B + theta_C
return Pose3D(np.array([[x_result], [y_result], [theta_result]]))
```

1.2.3 Inverse pose compounding

Inverse compounding expresses frame A relative to frame B from the pose of B relative to A:

$$ {}^{B}\boldsymbol{x}_A = \ominus {}^{A}\boldsymbol{x}_B = \begin{bmatrix} -{}^{A}x_B\cos({}^{A}\psi_B) - {}^{A}y_B\sin({}^{A}\psi_B) \\ {}^{A}x_B\sin({}^{A}\psi_B) - {}^{A}y_B\cos({}^{A}\psi_B) \\ -{}^{A}\psi_B \end{bmatrix}. \tag{5} $$

This is the coordinate form of $T({}^{B}\boldsymbol{x}_A) = T({}^{A}\boldsymbol{x}_B)^{-1}$.

The implementation evaluates equation (5) directly and returns the inverse pose as another `Pose3D` object:

Listing 2: Core of the inverse compounding implementation.

```
x_result = -xB * cos(theta_B) - yB * sin(theta_B)
y_result =  xB * sin(theta_B) - yB * cos(theta_B)
theta_result = -theta_B
return Pose3D(np.array([[x_result], [y_result], [theta_result]]))
```

After implementing and testing both operators in the Pre-Lab notebook, the same code was included in `Pose3D.py`. This makes $\oplus$ and $\ominus$ available as `Pose3D` methods for the differential-drive simulation developed in Lab 1.

2 Lab 1: Simulation

Lab 1 consists of completing the `DifferentialDriveSimulatedRobot` class, which inherits from `SimulatedRobot`. The base class provides the common simulation interface, state storage, and visualisation tools, whereas the derived class implements the differential-drive motion and sensor models.

2.1 `__init__`: robot configuration

The constructor first calls `super().__init__` to initialise the state, map, sampling period, and visualisation variables provided by `SimulatedRobot`. The simulated state contains the world-frame pose and body-frame velocity:

$$\boldsymbol{x}_{s,k} = \begin{bmatrix} \boldsymbol{\eta}_{s,k} \\ \boldsymbol{\nu}_{s,k} \end{bmatrix} = \begin{bmatrix} x_k & y_k & \psi_k & u_k & v_k & r_k \end{bmatrix}^T. \tag{6}$$

It then defines the parameters required by the differential-drive model. The wheel separation is $b = 0.5$ m, the wheel radius is $R = 0.1$ m, and each wheel revolution produces $N_p = 1024$ encoder pulses. The motion and encoder noise are described by

$$Q_s = \text{diag}\left(0.1^2,\, 0.01^2,\, \text{deg2rad}(1)^2\right), \qquad R_e = \text{diag}(22^2, 22^2). \tag{7}$$

Here, Q_s is the covariance of the acceleration perturbation applied by the motion model, while R_e is the covariance of the encoder noise in squared pulses. The remaining constructor parameters configure sensors that are not required for the simulation and dead-reckoning tasks discussed here.

2.2 `fs`: motion model

This method computes the new simulated state from the previous state and the commanded forward and angular velocities. The desired input is $\boldsymbol{u}_{s,k} = [u_d,\, r_d]^T$. Because a differential-drive robot cannot move laterally, the corresponding desired body velocity is $\boldsymbol{\nu}_d = [u_d,\, 0,\, r_d]^T$.

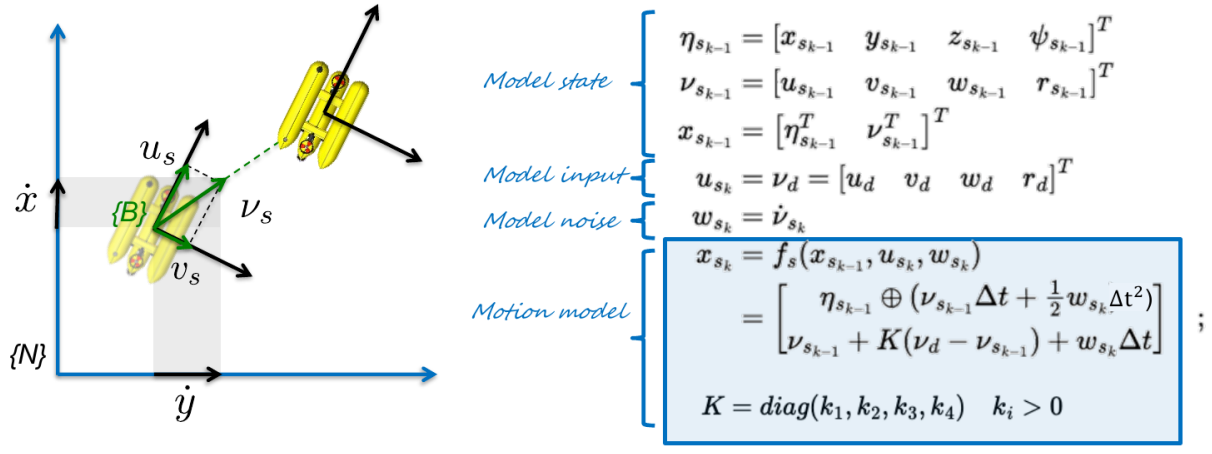


Figure 1: State, input, noise, and motion model used by `fs`.

The planar version of the model in figure 1 uses $\boldsymbol{\eta} = [x,\, y,\, \psi]^T$ and $\boldsymbol{\nu} = [u,\, v,\, r]^T$, with $v_d = 0$ because the wheels prevent lateral motion.

The nominal implementation uses $K = I_3$. Its effect on the velocity response is discussed in section 4.

The key lines in `fs` are therefore:

Listing 3: Pose and velocity update in the simulator.

```
w = np.random.multivariate_normal(np.zeros(3), self.Qsk).reshape(3, 1)
eta_new = eta_prev.oplus(Pose3D(
    nu_prev * self.dt + 0.5 * w * self.dt**2))
nu_new = nu_prev + K @ (nu_desired - nu_prev) + w * self.dt
```

2.3 `ReadEncoders`: wheel-encoder simulation

`ReadEncoders` converts the motion between two consecutive simulated poses into the measurements that would be produced by the left and right wheel encoders. The implemented sequence is summarised in figure 2.

1. Relative motion in the previous body frame

$$\Delta\boldsymbol{\eta}_k = (\ominus\boldsymbol{\eta}_{s,k-1}) \oplus \boldsymbol{\eta}_{s,k}$$

↓

2. Extract the local forward displacement and yaw change

$$d = \Delta x_k, \qquad \Delta\psi = \Delta\psi_k$$

↓

3. Compute the distance travelled by each wheel

$$d_L = d - \frac{b}{2}\Delta\psi, \qquad d_R = d + \frac{b}{2}\Delta\psi$$

↓

4. Convert wheel distances into ideal encoder pulses

$$\boldsymbol{z}_{\text{ideal}} = \frac{N_p}{2\pi R}\begin{bmatrix} d_L \\ d_R \end{bmatrix}$$

↓

5. Add encoder noise and form the measurement

$$\boldsymbol{\epsilon}_k \sim \mathcal{N}(\mathbf{0}, R_e), \qquad \boldsymbol{z}_{s,k} = \boldsymbol{z}_{\text{ideal}} + \boldsymbol{\epsilon}_k$$

Figure 2: Sequence implemented by `ReadEncoders`.

The use of $\ominus$ in the first step is essential. Both simulated poses are stored relative to the world frame, but an encoder measures the displacement in the robot frame. In transformation form, the required relative motion is

$$T(\Delta\boldsymbol{\eta}_k) = T(\boldsymbol{\eta}_{s,k-1})^{-1} T(\boldsymbol{\eta}_{s,k}). \tag{8}$$

The inverse operator removes the previous world-frame pose, and the following $\oplus$ expresses the current pose relative to that previous robot frame. A direct subtraction would not rotate

the translational displacement into the body axes and would therefore give incorrect wheel motion when the robot is not aligned with the world frame.

For the implemented robot, $b = 0.5$ m, $R = 0.1$ m, and $N_p = 1024$. The encoder covariance is $R_e = \mathrm{diag}(22^2, 22^2)$ pulses^2. The method returns $(\boldsymbol{z}_{s,k}, R_e)$, where $\boldsymbol{z}_{s,k} = [n_L, n_R]^T$ contains the noisy left and right encoder pulses and R_e describes their measurement uncertainty.

2.4 Compass and robot visualisation

`ReadCompass` adds zero-mean Gaussian noise to the true yaw and returns the resulting measurement together with its variance. `PlotRobot` updates the vehicle icon using the current simulated pose.

Listing 4: Essential compass and visualisation operations.

```
yaw_true = self.xsk[2, 0]
yaw_measured = yaw_true + np.random.normal(0.0, self.v_yaw_std)
return yaw_measured, self.v_yaw_std**2

self.vehicleIcon.update([self.xsk[0], self.xsk[1], self.xsk[2]])
plt.pause(0.0000001)
```

2.5 Trajectory commands

The simulator is tested with a constant forward speed and opposite angular velocities. The first command produces circular motion; changing the angular velocity sign produces the second lobe of the requested figure-eight sequence. Each command is applied for approximately one complete revolution, $k_{\mathrm{steps}} = \mathrm{round}(2\pi/(0.03\Delta t)) = 2094$. The applied commands are $(u_d, r_d) = (0.5, 0.03)$ and $(0.5, -0.03)$. The second command starts from the final simulated state of the first, preserving continuity between the two trajectory segments.

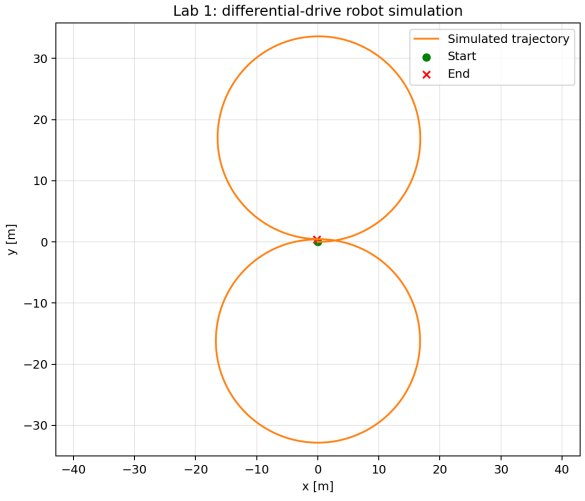


Figure 3: Trajectory generated directly by the simulated robot for the two opposite angular-velocity commands.

3 Lab 2: Dead-Reckoning Localization

3.1 Localisation sequence

The `Localization` base class defines the common iterative procedure. At every time step it:

1. advances the simulated robot through `robot.fs`;
2. obtains the encoder-derived input through `GetInput`;
3. updates the pose estimate through `Localize`; and
4. plots the estimated trajectory.

The derived `DR_3DOFDifferentialDrive` class supplies the two algorithm-specific methods in steps 2 and 3.

3.2 `GetInput`: encoder readings to body motion

The template documentation describes the input both as the raw encoder vector $[n_L, n_R]^T$ and as a generic four-component velocity. Neither is the value that `Localize` ultimately requires in this planar implementation. `GetInput` therefore reads $[n_L, n_R]^T$ from the simulated robot and converts it into the three-component body velocity $\boldsymbol{u}_k = \boldsymbol{\nu}_k = [u_k, 0, r_k]^T$.

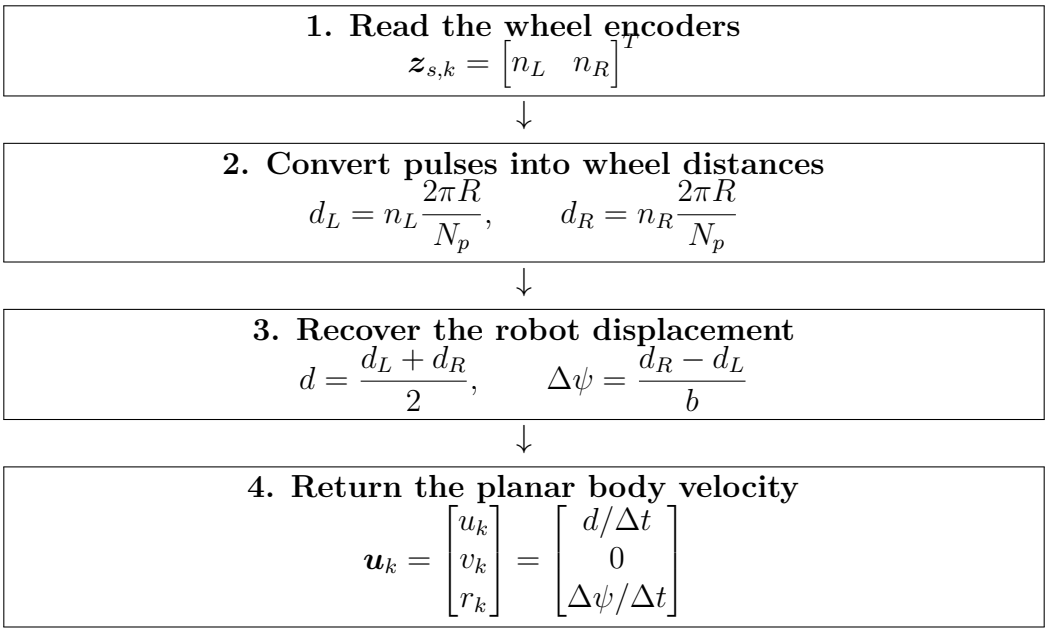


Figure 4: Conversion performed by `GetInput`.

The zero lateral component represents the non-holonomic constraint of the differential-drive platform. The implementation uses $N_p = 4096$ pulses per wheel revolution, $R = 0.1$ m, $b = 0.5$ m, and $\Delta t = 0.1$ s.

3.3 Dead-reckoning pose update

The estimator integrates the encoder-derived velocity over one sampling period and compounds that body-frame displacement with the previous estimate:

$$\widehat{\boldsymbol{\eta}}_k = \widehat{\boldsymbol{\eta}}_{k-1} \oplus (\boldsymbol{\nu}_k \Delta t)\,. \tag{9}$$

The multiplication by Δt converts velocity into a local pose increment. The $\oplus$ operation then rotates its translation according to the previous heading before updating the world-frame pose. The complete method is:

Listing 5: Complete `Localize` implementation.

```
def Localize(self, xk_1, uk):
    self.etak_1 = xk_1
    self.uk = uk
    nu_k = uk

    etak = Pose3D(xk_1).oplus(nu_k * self.dt)
    return etak
```

No compass, landmark or absolute-position measurement is used in this update. Consequently, every encoder error is integrated into the next pose and cannot be removed by the estimator.

4 Results and discussion

The comparisons use the same figure-eight command, 2094 steps per turn, and a fixed random seed. Only the parameter indicated in each test is changed.

4.1 Effect of the velocity gain

The first test uses $K = kI_3$ with $k = 0.25$, 0.5, 1, and 1.5, while the encoder noise remains at its nominal value of 22 pulses. For each value, figure 5 shows the first turn and the complete figure-eight.

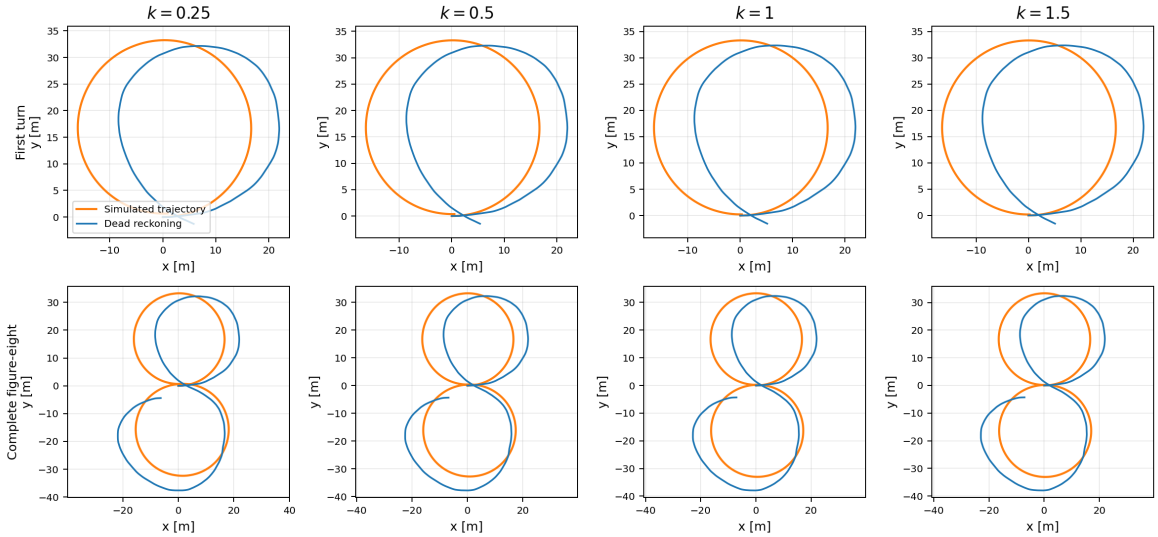


Figure 5: Simulated and dead-reckoning trajectories for the four tested velocity gains.

The four cases produce very similar trajectories, so changing K has less effect than the encoder noise in this test. The lower gains leave a slightly less regular simulated path after changing the turn direction, whereas $k = 1$ and $k = 1.5$ give almost the same result. We keep $K = I_3$ because it follows the command directly without using a gain greater than one.

4.2 Effect of the encoder noise

The second test fixes $K = I_3$ and repeats both trajectories with $\sigma_e = 0$, 11, 22, and 44 pulses. The same noise sequence is used in every case and only its amplitude is changed.

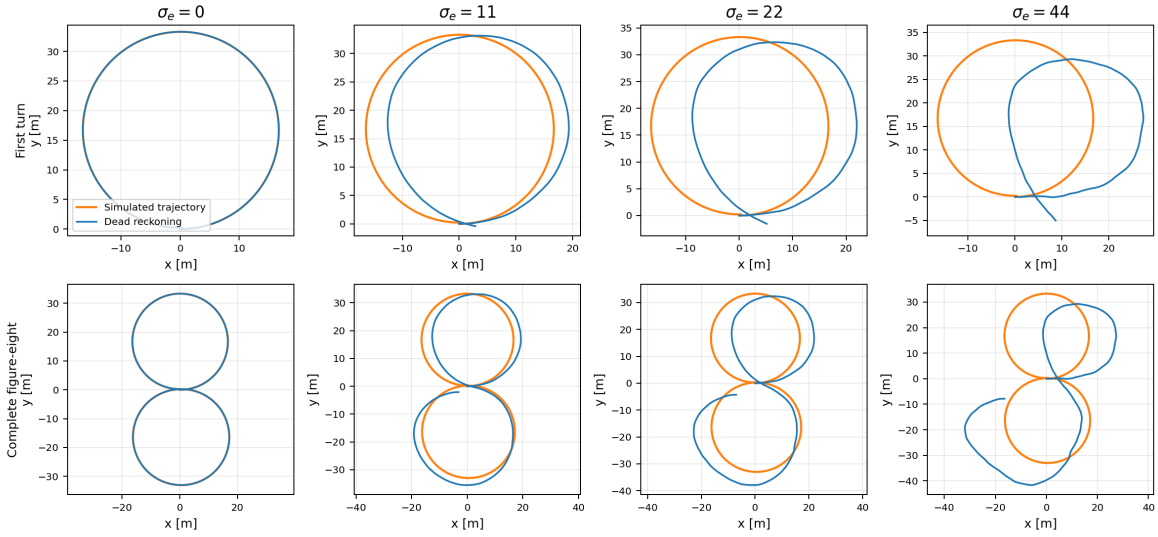


Figure 6: Simulated and dead-reckoning trajectories for the four tested encoder-noise levels.

With $\sigma_e = 0$, the simulated and estimated trajectories practically overlap. A small

separation appears with 11 pulses, becomes clear at the nominal value of 22 pulses, and strongly deforms the estimate at 44 pulses. The smallest noise gives the closest result, while 22 pulses is retained as the nominal sensor setting. This comparison also shows that encoder noise has a much clearer influence on dead reckoning than the tested values of K.

5 Conclusions

The two laboratories establish a complete chain from planar rigid-body kinematics to simulated sensing and odometric localisation. The `Pose3D` class provides consistent direct and inverse compounding, which is then reused by both the simulator and estimator. The differential-drive simulator generates noisy motion and encoder pulses, while the dead-reckoning class converts those pulses into incremental body motion and propagates the pose estimate. Circular and opposite-turn tests verify the expected robot motion and also demonstrate the fundamental limitation of dead reckoning: without an absolute observation, encoder noise produces unbounded drift.